



String Variable

🕒 Created time	June 20, 2026 1:06 PM
☰ Category	Variables
👤 Last edited by	 R.A R.A
🕒 Last updated time	June 28, 2026 7:22 PM

```
string firstName = string.Empty;
string lastName = string.Empty;
string filePath = string.Empty;

firstName = "Tim";
lastName = "Corey";
//filePath = "C:\\Temp\\Demo";
filePath = @"C:\Temp\Demo";
//firstName = "123";

//Console.WriteLine(firstName + " " + lastName);

string testString = $"The file for{firstName} is at C:\SampleFile";

//string interpolation
Console.WriteLine($"Hello{firstName}{lastName}");
Console.WriteLine(filePath);
Console.WriteLine(testString);
```

1. אתחול משתנים

```
string firstName = string.Empty;
string lastName = string.Empty;
string filePath = string.Empty;
```

במקום לאתחל מחרוזת בעזרת מרכאות ריקות (""), משתמשים ב- `string.Empty`.
זהו הסטנדרט המקצועי כיום: זה הרבה יותר קריא לעין, מונע שגיאות (כמו להשאיר רווח בטעות בין המרכאות),
ויותר יעיל מאחורי הקלעים מבחינת הקצאת זיכרון.

2. השמת ערכים

```
firstName = "Tim";
lastName = "Corey";
```

כאן אנחנו מכניסים את נתוני הטקסט לתוך המשתנים שיצרנו.

3. מחרוזת מילולית (Verbatim String - @)

```
//filePath = "C:\\Temp\\Demo";
filePath = @"C:\Temp\Demo";
```

בשורה המוקדמת (הראשונה), כדי לכתוב לוכסן שמאלי (\), היינו צריכים לכתוב אותו פעמיים, כי ב-C# לוכסן בודד נחשב ל"תו בריחה" (Escape Character) שמשמש לדברים כמו ירידת שורה (\n).

כדי לפתור את הסרבול הזה, הוסיפו את השטרודל (@) לפני תחילת המחרוזת, שאומר לקומפיילר לקרוא את הטקסט בדיוק כמו שהוא כתוב, ולא לחפש תווים מיוחדים. זו הדרך המקובלת והנקייה ביותר לכתוב נתיבים של קבצים.

4. שרשור מודרני (String Interpolation - \$)

```
string testString = $"The file for {firstName} is at C:\SampleFile";
```

כאן יש שילוב של שני כוחות חזקים:

- **הסימן \$**: מאפשר לנו לשתול משתנים (כמו {firstName}) ישירות בתוך המחרוזת. פעם היו מחברים מחרוזות עם סימן פלוס (+), אבל היום זה נחשב לפרקטיקה מיושנת ופחות יעילה. `Console.WriteLine(firstName + " " + lastName);` - שורה בהערה בכוונה כדי להראות איך לא לעשות את זה היום.
- **הסימן @**: משאיר את היכולת לכתוב את נתיב הקובץ (C:\SampleFile) בצורה חלקה בלי להכפיל לוכסנים.

5. הדפסה לקונסול

```
Console.WriteLine($"Hello {firstName} {lastName}");
Console.WriteLine(filePath);
Console.WriteLine(testString);
```

כאן אנחנו מדפיסים את התוצאות למסך.

בשורה הראשונה יש שימוש נקי ב-String Interpolation (\$) כדי לשלב את השם הפרטי ושם המשפחה בתוך ההדפסה.

~~~~~

## 1. מכיל כל מידע שנרצה

המשתנה (**Variable**) כמו קופסת אחסון בזיכרון של המחשב. ברגע שירנו את הקופסה הזו, אפשר להכניס לתוכה כל נתון שנרצה, לשנות אותו במהלך ריצת התוכנית, או לשלוף אותו מתי שנצטרך, זה המקום לשמור את המידע של האפליקציה.

## 2. כל עוד זה מהסוג הנכון

כאן נמצא הקאצ', שפת #C היא שפה קפדנית מאוד (המונח המקצועי הוא **Strongly Typed**), זה אומר שכשיוצרים את הקופסה, חייבים להדביק עליה תווית שמגדירה מראש איזה סוג של מידע מותר להכניס פנימה.

### במקרה של מחרוזת (String):

- אם הגדרנו שהמשתנה הוא מסוג `string`, הוא יקבל בשמחה כל טקסט שנכניס אליו (למשל `"Tim"` או `"Hello"`).
- אבל, אם ננסה פתאום להכניס לאותה קופסה מספר מתמטי טהור (כמו `123` ללא מרכאות), המחשב מיד יסמן קו אדום ויסרב לקמפל את הקוד, הקופסה הוגדרה לטקסט בלבד, ואי אפשר "לרמות" אותה ולהכניס סוג אחר של נתונים.

המשתנה הוא הגמיש ביותר מבחינת התוכן שניתן לשים בו, אבל נוקשה לחלוטין מבחינת הסוג (Type) של אותו תוכן.

```
string firstName = string.Empty;  
  
string myFirstName = string.Empty;  
  
string test = string.Empty;
```

אחת ממוסכמות הכתיבה הבסיסיות והחשובות ביותר ב-#C, שנקראת **Camel Case** (שיטת ה"גמל" - כי האותיות הגדולות באמצע המילה מזכירות דבשות של גמל).

### הכללים המדויקים למתן שמות למשתנים:

## 1. משתנה המורכב ממספר מילים

מכיוון שאסור להשתמש ברווחים בשם של משתנה, השיטה מחברת את המילים יחד בצורה שעדיין קלה לקריאה לעין:

- המילה הראשונה נכתבת תמיד באותיות **Lower case**.
- האות הראשונה של כל מילה נוספת שמגיעה אחריה נכתבת תמיד באות **Capitalized**.
- בדוגמאות:** \* `firstName` (ה- `f` קטנה, וה- `N` גדולה ומציינת את תחילת המילה השנייה).
  - גדולות כדי להפריד את המילים הבאות `N`-וה `F`-קטנה, אבל ה `m`-ה) `myFirstName`.

## 2. משתנה המורכב ממילה אחת בלבד

**הכלל כאן הוא הפשוט ביותר:** אם השם שבחרנו למשתנה מכיל רק מילה אחת, כל המילה כולה תיכתב תמיד באותיות קטנות בלבד.

- **בדוגמה:** \* `test` (המילה כולה מורכבת מאותיות קטנות, ללא שום אות גדולה).

המטרה של השיטה הזו היא ליצור קוד אחיד ומקצועי בכל רחבי הפרויקט, כך שכל מי שיקרא אותו יוכל לזהות מיד ובקלות שמדובר בשם של משתנה.

~~~~~

```
string a = string.Empty;
string b = string.Empty;
string c = string.Empty;

string b = string.Empty;
string a = string.Empty;
string c = string.Empty;

string firstNameForTheUserWhoIsUsingOurApplication = string.Empty;

string fName = string.Empty;
```

הקטע הזה נוגע באחד העקרונות הכי חשובים היום בתעשיית ההייטק:

קוד נקרא על ידי בני אדם הרבה יותר ממה שהוא נכתב, ולכן הקריאות שלו היא קריטית.

1. הפדיחה של אתר Experts Exchange (חשיבות ה-Camel Case)

הסיפור האמיתי והמצחיק על אתר בשם "Experts Exchange".

מכיוון שב-URL קשה להבחין באותיות גדולות ואין רווחים, הכתובת שלהם נכתבה ברצף אחד: `expertsexchange.com`. הבעיה היא שגולשים הפסיקו לראות את המילים Experts ו-Exchange, והמוח שלהם חילק את הרצף לשלוש מילים אחרות לגמרי: `expert sex change` (שינוי מין למומחים).

האתר ספג כל כך הרבה מבוכה שהוא נאלץ להוסיף מקף לכתובת (`experts-exchange`) רק כדי שאנשים יבינו על מה האתר.

- **מוסר השכל:** הדרך שבה אנחנו מדגישים תחילת מילים (בעזרת אותיות גדולות ב-Camel Case) מצילה אותנו מאי הבנות קריטיות.

2. לא להשתמש באותיות בודדות (a, b, c) כשמות של משתנים

```
string a = string.Empty;  
string b = string.Empty;
```

הקוד הזה יעבוד מצוין ויקומפל, אבל אנחנו לא כותבים קוד רק למחשב, אם מתכנתים אחרים יפתחו את הקובץ הזה, הם לא יבינו ש-`a` מייצג את השם הפרטי ו-`c` את שם המשפחה, או בכלל להפך. בתעשייה, איסור חמור הוא להשתמש באותיות בודדות עבור משתנים רגילים שנושאים מידע חשוב.

3. הקיצון השני: לא להשתמש בשמות ארוכים ומוגזמים

מצד אחד לקרוא למשתנה באות אחת כמו `a` זה קצר מדי וגרוע, מצד שני לקרוא למשתנה במשפט שלם זה גרוע לא פחות.

לדוגמא:

```
string firstNameForTheUserWhoIsUsingOurApplication = string.Empty;
```

זה מסורבל, יוצר בלגאן בעיניים ומקשה על קריאה מהירה של הקוד, השם חייב להיות ברור ומסביר את עצמו, אבל לא חופר, השם `firstName` עושה את העבודה בצורה מושלמת.

4. הכלל נגד קריאה למשתנים בראשי תיבות וקיצורים

שימוש בקיצורי מילים כגון זה רע:

```
string fName = string.Empty;
```

מפני שהמוח האנושי לא בנוי לקרוא קיצורים ולתרגם אותם מיד בראש למילה המלאה. כשקוראים מאות שורות קוד, כל עצירה קטנה כזו מעכבת את ההבנה, אנחנו תמיד נעדיף לכתוב את המילה המלאה (`firstName`) כדי שהקריאה תהיה חלקה, כמו קריאת טקסט רגיל.

~~~~~

```
string firstName = string.Empty;  
  
string.Empty;  
  
string firstName = string.Empty;  
  
string firstName = "Tim";
```

### שלב ראשון והחשוב ביותר בחייו של משתנה: האתחול.

שתי גישות מקובלות להכנת משתנה מסוג מחרוזת (`String`) לפני שמתחילים להשתמש בו בתוכנית:

## 1. הדרך הסטנדרטית: יצירת משתנה ריק ( `string.Empty` )

כשכותבים `string firstName = string.Empty` , בעצם פוקדים על המחשב להכין משתנה שמיועד לטקסט ולקרוא לו `firstName` , אבל בשלב זה להשאיר ריק.

- **המשמעות:** המשתנה קיים ומוכן לעבודה, אבל אין בתוכו שום מידע.
- **עושים זאת משום** שזו הדרך המסורתית והמקצועית ב-C# לסמן למערכת (ולמתכנתים אחרים שיקראו את הקוד) שרק התחלנו.
- אנחנו חייבים לתת למשתנה ערך התחלתי כלשהו כדי שהקומפיילר לא יזרוק שגיאה, אבל אנחנו מצהירים בצורה מפורשת שאת התוכן האמיתי נכניס אליו בשלב מאוחר יותר בקוד.

## 2. הדרך הישירה: הצבת הערך מיד והפחתת שורות קוד

אם אנחנו כבר יודעים מראש מה יהיה התוכן של המשתנה בעת היצירה שלו, אין שום היגיון ליצור אותו ריק בשורה אחת, ואז למלא אותו בשורה אחרת.

**במקום זה, עדיף לכתוב ישר:** `string firstName = "Tim"` .

- **המשמעות:** יצרנו את המשתנה, ובאותה נשימה כבר הכנסנו לתוכו את הערך האמיתי שלו ( `"Tim"` ).
  - **היתרון:** זה חוסך לנו שורות קוד מיותרות, מקטין את הבגאן בעיניים, והופך את הקובץ לקריא ויעיל יותר.
- אם לא יודעים מה יהיה הערך של המשתנה ברגע היצירה שלו (למשל, אם מחכים שהמשתמש יקליד משהו) - נאתחל אותו עם `string.Empty` .**
- אם יודעים מה הערך ההתחלתי שלו - נכניס אותו מיד באותה השורה.**

דרך נוספת לאתחל משתנה מסוג טקסט (**String**) כשאין לנו עדיין מידע להכניס לתוכו:

```
string firstName = "";
```

- **יצירת מחרוזת ריקה:** ברגע שפותחים וסוגרים מירכאות כפולות ( `""` ) בלי לכתוב שום מילה, אות או אפילו רווח ביניהן, בעצם מייצרים אובייקט טקסט חוקי לחלוטין, שהאורך שלו הוא פשוט אפס תווים.
  - **הקופסה ריקה, אבל קיימת:** המשתנה `firstName` נוצר ומוכן לעבודה, אך בשלב זה לא מאוחסן בו שום תוכן.
  - **הקשר ל- `string.Empty` :** מבחינת המחשב ומבחינת שפת C#, לכתוב `""` זה בדיוק אותו הדבר כמו לכתוב `string.Empty` , התוצאה מאחורי הקלעים זהה לחלוטין.
- יש מפתחים שמעדיפים לכתוב `""` כי זה קצר ומהיר יותר להקלדה, בעוד שאחרים מעדיפים את `string.Empty` כי זה נראה קריא וברור יותר לעין, זה עניין של סגנון אישי, ושתי הדרכים מקובלות ותקינות לחלוטין.

## שלב שבו אנחנו עוברים מהגדרות להכנסת מידע לתוך המשתנה והשימוש בו.

## 1. הכנסת הערך לקופסה

בשורה `firstName = "Tim";`, אנחנו פונים למשתנה שלנו (הקופסה שנקראת `firstName`) ומכניסים לתוכו בפועל את הטקסט `"Tim"`.

- מרגע זה, המשתנה שומר בתוכו את המידע שהכנסנו אליו.
- אין כאן את המילה `string` בהתחלה, הסיבה היא שהמחשב כבר מכיר את המשתנה מהשורה הקודמת שבה יצרנו אותו, ולכן אין צורך להצהיר על הסוג שלו מחדש, פשוט קוראים לו בשמו ומכניסים את הערך.

## 2. שליפת המידע והדפסתו

```
Console.WriteLine(firstName);
```

אנחנו פוקדים על המחשב לקחת את המידע ולהציג אותו על גבי הקונסול.

- **הנקודה הקריטית:** המחשב לא מדפיס את המילה `"firstName"`, מכיוון שאין מירכאות סביב המילה בתוך הסוגריים וזה אומר שהיא לא `string`, המחשב מבין שמדובר במשתנה, הוא ניגש לזיכרון, "פותח את הקופסה", שולף משם את התוכן שלה, ומדפיס אך ורק את התוכן.
- התוצאה הסופית שתודפס על המסך תהיה פשוט המילה: `Tim`.

~~~~~

```
firstName = "123";
```

אחת מנקודות ההבנה החשובות ביותר למפתחים בתחילת הדרך: ההבדל בין מספר אמיתי לבין טקסט שרק נראה כמו מספר.

1. כוחן המוחלט של המירכאות

- בשפת C#, ברגע שעוטפים משהו במירכאות כפולות (`""`), משנים לחלוטין את הזהות שלו בעיני המחשב.
- למרות שהעין האנושית שלנו קוראת את `123` כמספר טבעי, ברגע ששמנו אותו בתוך מירכאות `"123"`, המחשב רואה אך ורק רצף של תווים טקסטואליים, `string`.
 - המירכאות הן החוק הקובע, כל מה שנכנס לתוכן, אפילו אם אלו רק ספרות, מומר מיד ל-`string`.

2. המשמעות בפועל (איבדנו את המתמטיקה)

- מכיוון ש-`"123"` מוגדר כעת כטקסט ולא כמספר מתמטי (`Type שנקרא int`), אי אפשר לבצע עליו פעולות חשבון.
- אם ננסה לחבר, לחסר או להכפיל את `"123"`, הקומפיילר יזרוק שגיאה, מבחינת המחשב, ניסיון לעשות חישוב על `"123"` הוא חסר היגיון בדיוק כמו לנסות להכפיל את המילה `"Tim"`.
- זה שימושי לשמירת נתונים שמורכבים מספרות, אבל לא נועדו לחישובים מתמטיים לעולם.
- דוגמאות קלאסיות בתעשייה:** מספר תעודת זהות, מספר טלפון, או מיקוד, כולם נשמרים כ-`string` (בתוך מירכאות) ולא כמספרים.

```
string firstName = string.Empty;

firstName = "Tim";
firstName = "123";

Console.WriteLine(firstName);
```

שני עקרונות ליבה קריטיים של ניהול זיכרון ב-#.NET, שחובה להבין כדי לכתוב קוד יעיל בסטנדרטים של התעשייה היום: **Immutability** (העדר יכולת להשתנות) ואיסוף זבל.

1. מחרוזות אינן ניתנות לשינוי (String Immutability)

ב-#.NET, ברגע שמחרוזת נוצרת בזיכרון, אי אפשר לשנות את התוכן שלה.

- שורת הקוד `firstName = "Tim"`, המערכת הקצתה תא בזיכרון ושמה בו את הערך "Tim", המשתנה `firstName` פשוט מצביע על התא הזה.
- שורת הקוד `firstName = "123"`, המערכת לא דרסה את המילה "Tim" בתוך התא הקיים, במקום זאת, היא יצרה תא חדש לגמרי בזיכרון עבור "123", והעבירה את ההצבעה של המשתנה אליו.

2. אוסף הזבל

ברגע שההצבעה עברה ל-"123", התא הישן שהכיל את הערך "Tim" נשאר יתום ומרחף בזיכרון, כי שום משתנה בקוד כבר לא מקושר אליו. כאן נכנס לפעולה מנגנון של אוסף הזבל של סביבת ה-.NET, זוהי מערכת שעובדת ברקע באופן אוטומטי, היא סורקת את הזיכרון, מזהה אובייקטים יתומים שאין בהם יותר שימוש (כמו ה-"Tim" שלנו), ומנקה אותם כדי לפנות את משאבי הזיכרון למערכת.

3. מחרוזות נחשבות "יקרות"

מפני שהשמה של ערך חדש הורסת את הקשר לישן ומצריכה הקצאת זיכרון חדשה, תהליך יצירת אובייקט חדש בזיכרון ונטישת האובייקט הישן דורש כוח עיבוד (CPU) ומאלץ את אוסף הזבל של המערכת לעבוד קשה יותר כדי לנקות את השאריות, במערכות גדולות, התעסקות לא נכונה עם מחרוזות יכולה ליצור עומס מיותר ולפגוע בביצועים.

```
Console.WriteLine($"Hello {firstName} {lastName}");
"Hello {firstName} {lastName}"
```

טכניקה מודרנית ויומיומית ב-#.NET שנקראת **String Interpolation** (שילוב מחרוזות), והשפעה חשובה של הפעולה הזו על משאבי הזיכרון של המערכת.

- **הסימן דולר (\$)** בתחילת המחרוזת: הוספת ה-\$ לפני המרכאות אומרת לקומפיילר שזאת לא סתם מחרוזת טקסט פשוטה, שיהיה בה שילוב קוד C#."

- **הסוגיים המסולסלים ({ })**: בזכות ה-\$, כל מה שמוכנס בתוך הסוגיים המסולסלים מתפרש כקוד C# חי, לכן, במקום להדפיס את המילה המילולית "firstName", המערכת קוראת את המשתנים {firstName} ו-{lastName}, שולפת את הערכים האמיתיים ששמורים בהם באותו רגע, ושוללת אותם ישירות בתוך המשפט.

- **זה נחשב יקר למערכת** בגלל הסיבה שלמחרוזות יש העדר יכולת להשתנות, כשמתמשים ב-String **Interpolation** כדי לייצר את המשפט "Hello {firstName} {lastName}", המערכת לא מעדכנת מחרוזת קיימת ולא מחברת חלקים באותו מקום בזיכרון, המערכת לוקחת את כל החלקים ויוצרת עבורם **עותק חדש לחלוטין** בזיכרון.

המסקנה הטכנית כאן היא שכל מניפולציה על מחרוזת ב-C# מייצרת אובייקט חדש מאפס, ביצוע של הפעולה הזו פעמים בודדות הוא חסר משמעות מבחינת ביצועים, אבל אם הקוד עושה את זה אלפי פעמים ברצף (למשל בתוך לולאה שרצה מאחורי הקלעים), זה ייצר המון אובייקטים חדשים שיעמיסו על הזיכרון ויאצלו את אוסף האשפה לעבוד קשה יותר כדי לנקות אותם, מה שפוגע ביעילות ובמהירות של המערכת.

~~~~~

```
filePath = "C:\\Temp\\Demo \\n";
```

קומפיילר של C# מתמודד עם תווים מיוחדים בתוך מחרוזות (Strings), ומתמקד באתגר של כתיבת נתיבי קבצים עם הלוכסן השמאלי (Backslash - \).

- **סימנים מיוחדים המורים למחשב להתעלם מהמשמעות הרגילה של התו שאחריהם (Escape Characters):** בשפת C#, הלוכסן השמאלי (\) לא מתפרש כטקסט רגיל, אלא משמש כ-**Escape Character**, הוא מאותת למערכת שהתו שמגיע מיד עכשיו הוא פקודה מיוחדת, ולא טקסט, לדוגמה, השילוב n\ הוא פקודה שאומרת למערכת לבצע ירידת שורה .

- **השגיאה:** הנתיב "C:\\Temp\\Demo", המערכת זיהתה את T\ ואת d\ היא חיפשה פקודה מיוחדת שתואמת לאותיות האלו, וכיוון שאין פקודות כאלו ב-C#, התקבל קו אדום ושגיאה על רצף לא מוכר.

- **פתרון ראשון - שרשור לוכסנים (\\):** כדי לגרום למערכת להדפיס את הלוכסן עצמו מבלי לחשוב שזו פקודה, משתמשים בלוכסן כפול. הלוכסן הראשון הוא תו המילוט, שמפעיל את הלוכסן השני כטקסט פשוט. כך ניתן לכתוב את הנתיב ככה: "C:\\Temp\\Demo".

- **פתרון שני (והנפוץ בתעשייה) - מחרוזת מילולית (@):** הוספת סמל השטרודל (@) ממש לפני פתיחת המרכאות הופכת את המחרוזת ל-**String Literal**, זהו סימון שאומר לקומפיילר להתעלם מכל חוקי תווי ה-**Escape Characters**, וקרא את כל מה שבתוך המרכאות כטקסט פשוט בדיוק כפי שהוא מוקלד, שיטה זו נוחה מאוד לנתיבי קבצים, והנתיב יכתב בה כך: "@C:\\Temp\\Demo".

החיסרון היחיד הוא שבמצב כזה, אי אפשר להשתמש בפקודות כמו n\ בתוך אותה מחרוזת.

~~~~~

```
string testString = @"The file for {firstName} is at C:\SampleFile";
```

שילוב מתקדם, קריא ויעיל מאוד ב-C#: שימוש סימולטני ב-String Interpolation (\$) יחד עם Verbatim String Literal (@).

הטכניקה:

- **השילוב:** הוספת שני הסימנים יחד @\$ (או @\$) לפני פתיחת המרכאות מאפשרת ליהנות מהיתרונות של שתי השיטות באותה מחרוזת, מבלי להתפשר.
- **היתרון של ה-\$:** מאפשר לשתול משתנים חיים בתוך המחרוזת, במקרה הזה, המערכת מתייחסת ל-`{firstName}` כקוד לכל דבר, ותתרגם אותו לערך האמיתי שלו בזמן ריצה.
- **היתרון של ה-@:** מאפשר לכתוב תווים מיוחדים (כמו הלוכסן בנתיב `C:\SampleFile\`) כטקסט פשוט (Literal). הקומפילר מתעלם מתווי מ-Escape Characters, כך שאין צורך לסרב את הקוד עם לוכסנים כפולים (\\). הקוד מייצר מחרוזת חכמה שגם יודעת להזריק משתנים דינמיים וגם קוראת נתיבי קבצים באופן מילולי ונקי, זוהי הדרך המודרנית, והנפוצה ביותר היום בתעשייה לכתוב מחרוזות מורכבות הכוללות נתיבי קבצים, כתובות URL או טקסטים מרובי שורות.

```
string strFirstName = string.Empty;
```

מוסכמת שמות ישנה בשם **Hungarian Notation** (כתיב הונגרי), מדוע כיום בתעשייה המודרנית לחלוטין לא בשימוש.

- **מה הוא Hungarian Notation** זוהי שיטה שבה נהגו להוסיף קידומת לשם המשתנה כדי לציין איזה טיפוס נתונים (Type) הוא מחזיק, בדוגמה הוספת הקידומת `str` לפני המילה `FirstName` נועדה לסמן למתכנת שהמשתנה `strFirstName` הוא מסוג `String`.
- **החיכוך המנטלי:** המוח האנושי מתוכנת לקרוא מילים שלמות, כשאנחנו קוראים משתנה בשם `firstName`, המוח מבין את המשמעות מיד, לעומת זאת, כשאנחנו נתקלים ב-`strFirstName`, המוח לא מפצל את זה אוטומטית לשני חלקים שונים, המתכנת נאלץ לעצור לחלקיק שנייה, לפרק את המילה בראש, ולהבין את ההקשר.
- **ההשפעה על קריאות הקוד:** כשקוראים קוד של מערכת מלאה עם אלפי שורות ומשתנים, העצירות המנטליות הקטנות האלו מצטברות ומאטות מאוד את שטף העבודה, היום הכלל החשוב ביותר הוא **Clean Code** ושהקוד צריך להיות קריא וברור כמה שיותר קרוב לשפה טבעית.
- **היתרונות הטכניקה:** בעבר, לפני המצאת סביבות הפיתוח החכמות, הטכניקה הזו עזרה לזכור את סוגי המשתנים. היום, סביבות פיתוח מודרניות מציגות את סוג המשתנה אוטומטית על ידי צבעים או פשוט על ידי ריחוף עם העכבר מעל השם, לכן, הקידומת הזו הפכה למיותרת ורק עושה בלגאן בעיניים, כיום הסטנדרט הוא להשתמש בשמות נקיים כמו `firstName`.